

Alces JOB - Example STAGE1 (Requirements analysis/design/mockup)

alces-job will be a command-line tool that helps users quickly generate high-quality, ready-to-submit Slurm job scripts using templates, parameters, profiles, and site-specific defaults.

User Stories & Acceptance Criteria

Epic 1: Core Job Script Generation (MVP)

US-01: Generate a job script from command-line parameters

As a new or occasional HPC user,

I want to generate a complete Slurm job script by passing parameters on the command line,

so that I don't have to remember or copy-paste #SBATCH syntax.

Key functionality:

- Given no existing config, when I run the tool a valid .sh file is created containing correct #SBATCH directives.
- The generated script includes a shebang (#!/bin/bash), all requested #SBATCH lines, and the provided command.
- Output filename defaults to job-<job-name>.slurm (or user can override with -o).
- The tool exits with code 0 on success and prints a clear success message with the output filename.
- Invalid combinations produce a helpful error message and non-zero exit code.

US-02: Generate a job using a built-in template

As a researcher,

I want to generate a job script using a built-in template (e.g. mpi, gpu, array),

so that I get a correctly structured script with best-practice patterns for that job type.

Key functionality:

- Usage of the tool with more advanced options specified (eg MPI) produces a script that uses srun (or appropriate launcher) and sets environment variables correctly.
 - GPU options use a GPU-related setup.
 - Array options use a valid array setup.
 - Templates include helpful comments explaining key lines.
-

Epic 2: Interactive & Beginner-Friendly Experience

US-03: Interactive wizard mode

As a user new to Slurm,

I want an interactive mode that asks me questions step-by-step,

so that I can generate a correct job script without knowing all the flags.

Key functionality:

- Running `alces-job` with no arguments or `--interactive` starts a guided wizard.
 - The wizard asks for common fields in a logical order and provides sensible defaults.
 - It shows a preview of the generated script and asks for confirmation before writing the file.
 - Users can go back and change previous answers.
-

Epic 3: Workflow Integration

US-04: Submit job directly after generation

As a power user,

I want to generate and submit a job in one command,

so that I can iterate quickly without manually running `sbatch`.

Key functionality:

- `alces-job my-template --submit` generates the script, shows a preview, and then runs `sbatch`.
- The user is prompted for confirmation before submission (can be skipped with `--yes`).
- On success, it prints the submitted Job ID.
- `--dry-run --submit` shows what would be submitted without actually calling `sbatch`.

US-05: Validate an existing job script

As a user or HPC support staff,

I want to validate an existing Slurm script,

so that I can catch common mistakes before submission.

Key functionality:

- `alces-job validate job.sh` runs without error on a valid script.
 - It detects and reports common issues (missing shebang, duplicate directives, invalid time format, etc.).
 - It suggests improvements and can be used in CI pipelines (non-zero exit on errors).
-

Epic 4: Customization & Reuse

US-06: Save and reuse profiles

As a researcher running many similar jobs,

I want to save my common settings as a named profile,

so that I don't have to re-type the same flags every time.

Key functionality:

- `alces-job profile save my-gpu-experiment` creates a reusable profile.
- `alces-job --profile my-gpu-experiment` loads the saved settings (additional flags can still override them).
- Profiles are stored in `~/.config/alces-job/profiles/`.
- `alces-job profile list` shows all available profiles.

US-07: Create and use custom templates

As a power user or lab admin,

I want to create my own reusable templates,

so that my team can generate consistent, site-specific job scripts.

Key functionality:

- `alces-job template create my-lab-template` creates a new Jinja2-based template.
 - Custom templates support variables, conditionals, and loops.
 - User templates take precedence over built-in ones with the same name.
-

Epic 5: Usability & Documentation

US-08: Excellent help and examples

As a any user,

I want clear, contextual help and practical examples,

so that I can quickly understand how to use the tool.

Key functionality:

- `alces-job --help` shows well-organized help with examples.
- Subcommand help provides detailed flag descriptions and examples.
- The documentation includes complete examples for serial, MPI, GPU, and job array use cases.
- Shell completion works for commands, flags, and template names.

US-09: Edit an existing generated script

As a user who already has a script,

I want to quickly modify parameters of an existing script,

so that I don't have to regenerate everything from scratch.

Key functionality:

- `alces-job edit myjob.sh --time=4:00:00 --mem=16G` updates the relevant `#SBATCH` lines while preserving user-added content.
 - A backup of the original file is created (or the user is warned).
 - The tool shows a diff of changes before applying them.
-

Epic 6: Site Administration & Cluster Defaults

US-10: Provide cluster-wide defaults and guidance as an HPC administrator

As a HPC system administrator or research computing support staff,

I want to deploy a system-wide configuration for alces-job with our cluster's recommended defaults, valid partitions, typical resource limits, and site-specific modules/advice,

so that users on our cluster automatically get sensible, up-to-date defaults and are gently guided toward best practices.

Key functionality:

- The tool supports a system configuration file at `/etc/alces-job/config.yaml` that merges with (but can be overridden by) the user's personal config.
- Administrators can define:
 - Default partition(s) and fallbacks
 - Default account / QoS
 - Recommended vs allowed partitions (with optional warnings)
 - Default modules, output patterns, and resource guidance
 - Cluster-specific GRES types and features
- When users run `alces-job`, the tool automatically applies system defaults.
- The tool can emit soft warnings for non-recommended usage.
- Administrators can ship or override built-in templates.
- Generated scripts and `--help` can include a note about active site configuration.
- Clear documentation exists for deploying `alces-job` + system config across a cluster (Ansible, packaging, shared filesystem, etc.).
- Users can bypass site defaults with `--no-site-config`.

MOCKUP POSSIBILITY...

Usage: `alces-job [OPTIONS] [TEMPLATE]`

Generate high-quality Slurm job scripts from templates or command-line parameters.

`alces-job` helps you create correct, consistent, and best-practice Slurm batch scripts quickly — whether you're running a simple serial job or a complex multi-node GPU workload.

Options:

<code>-J, --job-name TEXT</code>	Job name (shown in queue)
<code>-N, --nodes INTEGER</code>	Number of nodes
<code>-n, --ntasks INTEGER</code>	Total number of tasks
<code>-c, --cpus-per-task INTEGER</code>	CPUs per task (for OpenMP/hybrid)
<code>--mem TEXT</code>	Memory per node (e.g. 8G, 16000M)

-t, --time TEXT	Walltime limit (e.g. 1-00:00:00 or 04:00:00)
-p, --partition TEXT	Partition / queue to use
-A, --account TEXT	Account / project to charge
--gres TEXT	Generic resources (e.g. gpu:1, gpu:a100:2)
-o, --output TEXT	Standard output file pattern
-e, --error TEXT	Standard error file pattern
--mail-user TEXT	Email address for notifications
--mail-type TEXT	Email notification types (BEGIN,END,FAIL,ALL)
--module TEXT	Module(s) to load (can be used multiple times)
--workdir TEXT	Working directory for the job
--command TEXT	Command or script to execute
--array TEXT	Job array specification (e.g. 1-100%10)
--dependency TEXT	Job dependencies
--profile TEXT	Load settings from a saved profile
-i, --interactive	Start interactive wizard mode
--submit	Generate and submit the job immediately
--dry-run	Show what would be generated/submitted
--no-site-config	Ignore system-wide site defaults
--help	Show this message and exit
--version	Show version and exit

Commands:

validate Validate an existing Slurm job script for common issues
edit Edit an existing job script (update resources, etc.)
profile Manage saved configuration profiles

template Create, list, or manage custom templates

Examples:

Simple serial job

```
alces-job --job-name=hello --time=00:30:00 --command "python hello.py"
```

MPI job using built-in template

```
alces-job mpi --nodes=4 --ntasks-per-node=8 --time=2:00:00 --submit
```

GPU job with specific resources

```
alces-job gpu --gres=gpu:a100:2 --mem=64G --time=12:00:00 my_script.sh
```

Interactive mode (great for beginners)

```
alces-job --interactive
```

Validate an existing script

```
alces-job validate myjob.slurm
```

Load a saved profile and override time

```
alces-job --profile my-gpu-experiment --time=8:00:00 --submit
```